ARRAYS



OUTLINE

- Basics of Arrays
- Declaring Arrays
- Initializing Arrays
- Accessing Arrays
- Processing Arrays

ARRAY BASICS

- *An array is used to store multiple values of the same type.*
  - *For example: we need to store the marks of 100 students.*
  - Instead of declaring individual variables, such as marks0, marks1, . . . , and marks99, you declare one array with a name such as marks

- *An element in an array can be accessed using an index.*
  - use marks[0], marks[1], . . . , and marks[99] to represent individual variables

DECLARING ARRAYS

- To declare an array, you need to specify its *element type (data type)* and size using the following syntax:

  elementType arrayName[SIZE];

- The **elementType** can be any data type, and all elements in the array will have the same data type.
- The **SIZE**, known as *array size declarator*, must be a constant integer greater than zero therefore, it is fixed.
  - *For example: we need to store the marks of 100 students; the syntax would be as follows:*

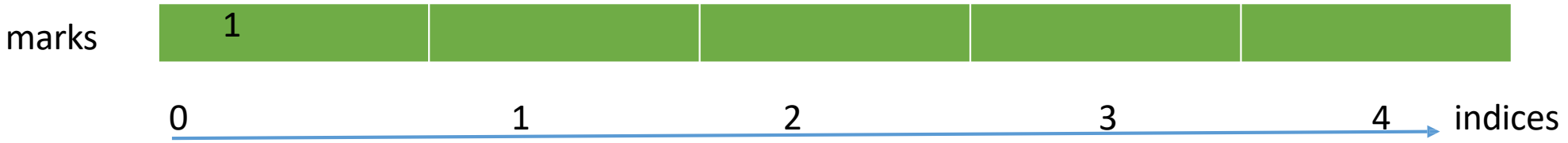
    int marks[100];

ACCESSING ARRAY ELEMENTS

- The array elements are accessed through the integer index.
- Array **indices are *0-based***; that is, they run from 0 to arraySize-1.
- The first element is assigned the index 0, the second element is assigned 1, and so on.
- For example we have an array: int marks[10];
  - ✓ marks[0]=10; means the value of first element of array = 10;
  - ✓ Similarly marks[9] = 20; means the value of last element of array = 20;
    - ❑ **Point to note: when we declare array such as int marks[10], the square brackets contain the size of the array while after declaration when we use this array, such as marks[4] than square brackets contain the position of element and marks[4] mean the element which lies at 5th position.**

REPRESENTATION OF AN ARRAY

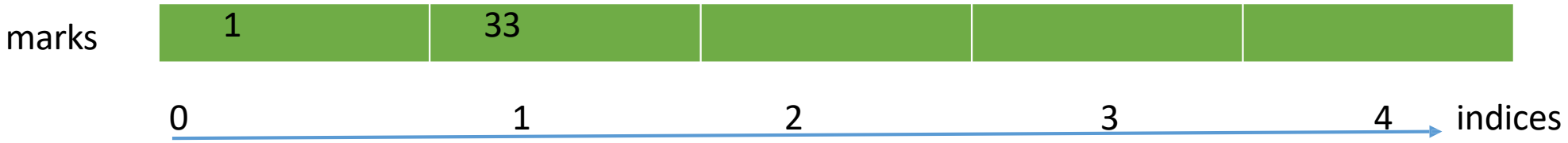
- int marks[5];



- marks[0] = 1;

REPRESENTATION OF AN ARRAY

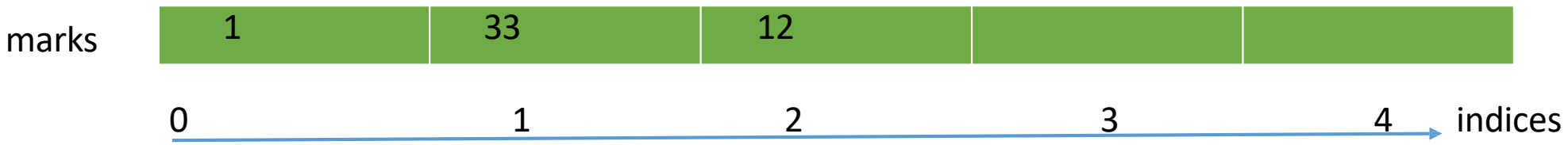
- int marks[5];



- marks[0] = 1;
- marks[1] = 33;

REPRESENTATION OF AN ARRAY

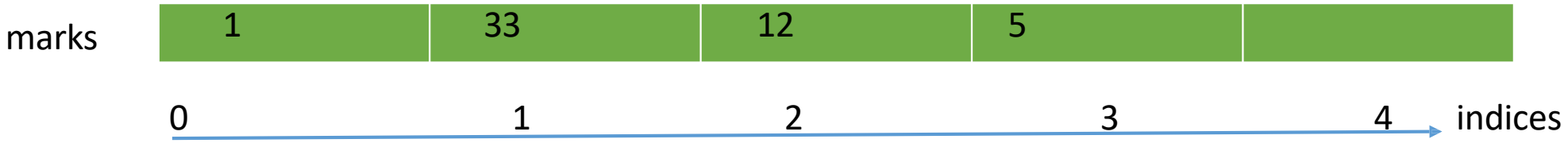
- int marks[5];



- marks[0] = 1;
- marks[1] = 33;
- marks[2] = 12;

REPRESENTATION OF AN ARRAY

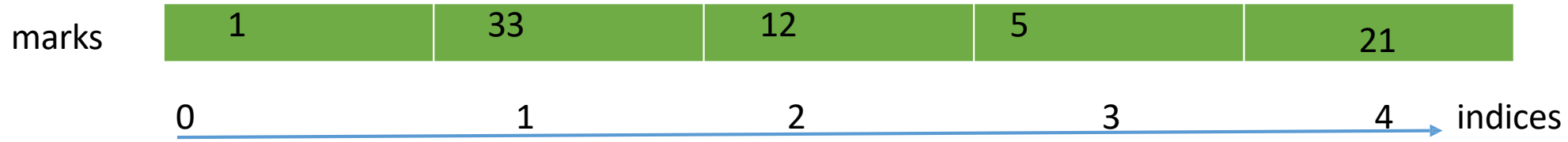
- int marks[5];



- marks[0] = 1;
- marks[1] = 33;
- marks[2] = 12;
- marks[3] = 5;

REPRESENTATION OF AN ARRAY

- int marks[5];



- marks[0] = 1;
- marks[1] = 33;
- marks[2] = 12;
- marks[3] = 5;

INITIALIZING ARRAYS

- int marks[4]= {1,2,3,4};
- C++ allows you to omit the array size when declaring and creating an array using an initializer
  - int marks[]= {1,2,3,4}; Size of array will be 4
- int marks[4];

```
marks[0]=1;
marks[1]=11;
marks[2]=33
marks[3]=4;
```

INITIALIZING ARRAYS

int marks[4]= {1,2};

- C++ allows you to initialize a part of the array. For example, the above statement assigns values 1, 2 to the first two elements of the array named marks. The other two elements will be set to zero.

Note: below syntax gives an error

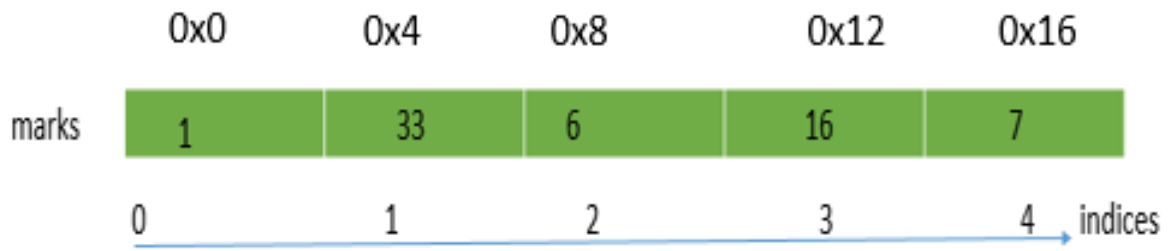
int marks[];

```
marks[0]=1;
marks[1]=11;
marks[2]=33
marks[3]=4;
```

Note that if an array is declared, but not initialized, all its elements will contain "garbage," like all other local variables.

MEMORY ALLOCATION

- The compiler allocates the space for 5 int type elements for array named marks.
- All the elements of the array are stored in Contiguous memory locations.



- The name of the array contains the address of 1st element of array.
  - ❑ cout<<marks;
  - ❑ Print 0x0 on the screen.

PROCESSING ARRAYS

- When processing array elements, you will often use a for loop—for the following reasons:
  1. All of the elements in an array are of the same type. They are evenly
     - processed in the same way using a loop.
  2. Since the size of the array is known, it is natural to use a for loop.
- *Initialing arrays with input values*: The following loop initializes the array marks with user input values:
  - int ARRAY_SIZE = 5;
  - for (int i = 0; i < ARRAY_SIZE; i++) cin >> marks[i];

Next Lecture

Two-Dimensional Arrays

C Strings